

Evaluating Text Classification Methods Using Neural Networks, BERT, and Llama-3 on the 20 Newsgroups Dataset

Cynthia Gathoni Gichuki

School of Computer Science and Information Technology

Lucerne University of Applied Sciences and Arts

Rotkreuz, Switzerland

cynthia.gichuki@stud.hslu.ch

<https://github.com/CynthiaGichuki/nlpProject>

Abstract—This paper presents a comprehensive comparative study of text classification methods applied to the 20 Newsgroups dataset, a benchmark corpus of 18,846 newsgroup posts spanning 20 topic categories. Eight experiments are conducted, ranging from classical word embedding and term-frequency approaches with a simple two-layer neural network, to transformer-based fine-tuning and large language model prompting. The methods evaluated include Word2Vec with mean-pooling, TF-IDF vectorization, TF-IDF with bigrams and Latent Semantic Analysis (LSA) compression, BERT fine-tuning, BERT with domain-adaptive Masked Language Modeling (MLM) pretraining, and Llama-3 under zero-shot, few-shot, and QLoRA fine-tuning settings. Results demonstrate that BERT with MLM pretraining achieves the highest accuracy of 0.710, confirming that domain-specific language adaptation improves classification performance. TF-IDF (0.636) outperforms Word2Vec (0.586) for this task, as topic-specific vocabulary provides stronger discriminative signal than averaged semantic vectors. Llama-3 zero-shot (0.255) significantly underperforms fine-tuned models, demonstrating that task-specific training is more decisive than raw model scale. All experiments are tracked using Weights and Biases (W&B). The project is available at: <https://wandb.ai/gichukicynthia-hochschule-luzern/nalapro-20newsgroups>. Code was developed with the assistance of Claude (Anthropic), as permitted under the course guidelines.

Index Terms—Text Classification, Word2Vec, TF-IDF, BERT, Masked Language Modeling, Llama-3, QLoRA, Transfer Learning, 20 Newsgroups, Domain Adaptation

I. INTRODUCTION

The automatic sorting of text documents into fixed topic categories is a core problem in Natural Language Processing (NLP). It serves as the basis for many real-world tools, including email spam filters, news organizers, medical record systems, and social media moderation filters.

The main difficulty here is not just about making a model that can predict the right category. The true challenge lies in the data representation itself. We have to find a way to turn unstructured human language into numbers that a machine learning model can actually process, while still keeping the meaningful differences that separate one topic from another.

This project examines eight different ways to handle multi-class text classification using the 20 Newsgroups dataset [1], which is a standard test set containing 18,846 posts from 20 online discussion forums. The categories cover a broad variety of topics, including computer hardware, sports, politics, religion, and science. Many of these categories share similar words, which makes the classification task a real challenge.

The main research question of this project is: how do traditional word representation methods compare to modern transformer models and large language models when the underlying neural network classifier remains exactly the same? By locking the classifier down to a simple setup of two linear layers with a ReLU activation, the experiments can directly measure how much the input representation itself changes the final performance. This deliberate choice creates a fair comparison between Word2Vec, TF-IDF, and LSA without any interference from differing model architectures.

The experiments move step-by-step from simple methods to more complex ones. Tasks 1a through 1d establish a strong traditional baseline. Tasks 2 and 3 bring in BERT [2], testing it first with standard fine-tuning and then with an added domain adaptation step. Task 4 and the bonus experiment move into advanced large language models, evaluating Llama-3 [3] both with and without parameter-efficient fine-tuning via QLoRA [4].

A central objective of this project is to understand the reasons why specific methods succeed or fail, rather than just tracking final numbers. The results section examines these outcomes in detail, along with takeaways regarding the practical realities of building NLP systems.

II. RELATED WORK

Text classification has a long history in NLP. Early approaches relied on Bag-of-Words (BoW) representations combined with classical machine learning models such as Naive Bayes [11] and Support Vector Machines (SVMs) [12]. While simple, these methods established that vocabulary-based fea-

tures are strong signals for topic classification, a finding confirmed by the TF-IDF results in this work.

The introduction of distributed word representations through Word2Vec [6] and GloVe [13] represented a significant shift: words could now be placed in a continuous vector space where semantic similarity corresponded to geometric proximity. However, a persistent limitation of these static embeddings is that every word has exactly one representation regardless of context. For instance, the word "bank" has the same vector whether appearing in "river bank" or "bank account". This context-insensitivity limits their expressiveness for nuanced classification tasks.

The transformer architecture [14], and BERT [2] in particular, addressed this limitation through bidirectional self-attention, producing context-dependent representations. Fine-tuning pretrained BERT models on downstream classification tasks became the dominant paradigm, consistently outperforming static embedding approaches on standard benchmarks. Gururangan et al. [10] further showed that domain-adaptive pretraining, which means continuing MLM training on domain-specific unlabeled text before fine-tuning, consistently improves performance. This directly motivates Task 3 in this project.

More recently, very large language models such as GPT-4, Llama-2, and Llama-3 [3] have demonstrated remarkable zero-shot and few-shot capabilities through in-context learning [15], raising the question of whether fine-tuning is still necessary. Brown et al. [15] showed that GPT-3 could perform many NLP tasks without any gradient updates, purely through prompt engineering. However, as this project demonstrates, zero-shot performance on fine-grained multi-class classification tasks remains substantially below task-specific trained models, a finding consistent with recent literature on the limitations of prompting for structured prediction tasks.

Parameter-efficient fine-tuning methods such as LoRA [16] and QLoRA [4] have emerged as practical solutions that combine the representational power of large models with the efficiency of targeted adaptation. By training only a small fraction of parameters, these methods achieve performance competitive with full fine-tuning at a fraction of the computational cost, as demonstrated by the QLoRA bonus experiment in this work.

III. DATASET AND PREPROCESSING

A. Dataset Overview

The 20 Newsgroups dataset [1] is a collection of approximately 18,846 newsgroup posts, partitioned into a training set of 11,314 documents and a test set of 7,532 documents across 20 balanced topic categories. The categories span four broad thematic domains:

- **Computer technology:** `comp.graphics`, `comp.os.ms-windows.misc`, `comp.sys.ibm.pc.hardware`, `comp.sys.mac.hardware`, `comp.windows.x`
- **Recreational:** `rec.autos`, `rec.motorcycles`, `rec.sport.baseball`, `rec.sport.hockey`

- **Science:** `sci.crypt`, `sci.electronics`, `sci.med`, `sci.space`
- **Talk and religion:** `alt.atheism`, `soc.religion.christian`, `talk.politics.guns`, `talk.politics.mideast`, `talk.politics.misc`, `talk.religion.misc`, `misc.forsale`

The dataset is loaded using the `scikit-learn` `fetch_20newsgroups` function with the `remove=("headers", "footers", "quotes")` parameter set. This preprocessing decision is critical and deliberately chosen. Newsgroup headers contain metadata such as the sender's email address and the newsgroup name itself. Without this parameter, a simple model could achieve near-perfect accuracy simply by reading the category name from the header, which would constitute a form of data leakage. Removing these fields forces every model to learn from the actual post content, making the evaluation scientifically valid and focused on generalization.

One notable challenge of this dataset is the overlap between certain topic pairs. Categories such as `talk.religion.misc` and `alt.atheism` share a large proportion of their vocabulary, as do `sci.crypt` and `talk.politics.misc` because both discuss encryption and government surveillance. This overlap partially explains why even the best-performing models struggle to perfectly distinguish these categories, as reflected in the per-class F1 scores discussed in Section VI.

B. Text Preprocessing (Task 1a)

For Word2Vec-based experiments, a five-step preprocessing pipeline is applied to each document. This pipeline reduces noise and vocabulary size while retaining the semantic content most relevant to topic classification.

- 1) **Lowercasing:** All text is converted to lowercase to eliminate case variation (e.g., "NASA" and "nasa" become the same token).
- 2) **Tokenization:** NLTK's `word_tokenize` (Punkt tokenizer) splits documents into tokens, correctly handling punctuation and contractions.
- 3) **Alphabetic filtering:** Only tokens where `t.isalpha()` is `True` are retained, removing numbers, URLs, and mixed alphanumeric strings common in newsgroup posts.
- 4) **Stop-word removal:** NLTK's 179-word English stop-word list removes high-frequency function words that carry no discriminative topic signal.
- 5) **Lemmatization:** The `WordNetLemmatizer` reduces tokens to their canonical dictionary form ("running" → "run"). Lemmatization is preferred over stemming because it always produces valid English words, ensuring that inflected forms of the same word contribute to a single, stable Word2Vec vector.

It is important to note that TF-IDF vectorization intentionally does not use this preprocessing pipeline. The

`TfidfVectorizer` from scikit-learn receives raw text strings and performs its own internal tokenization, which is optimized for the construction of the sparse term-document matrix. Applying the Word2Vec pipeline to TF-IDF would strip information that TF-IDF uses to compute term frequencies, and the internal tokenizer already handles many of the same normalization steps.

IV. METHODS

A. Fixed Neural Network Architecture

For all Task 1 experiments (1b, 1c, 1d), the neural network architecture is held constant as specified by the assignment. The model consists of exactly two linear layers with a single ReLU activation function in between, implemented in PyTorch as:

$$\text{Linear}(\text{input_dim}, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 20)$$

The hidden dimension of 256 is fixed across all three experiments. Only the input dimension changes depending on the vectorization method: 100 for Word2Vec, 5,000 for TF-IDF, and 300 for the LSA experiment. The output dimension is always 20, corresponding to the 20 newsgroup categories.

The hidden dimension of 256 balances capacity and generalization. Training uses Adam [5] ($\text{lr}=0.001$, batch size 64, 20 epochs) with `CrossEntropyLoss`. The ReLU activation $f(x) = \max(0, x)$ is necessary to introduce non-linearity: without it, the two linear layers collapse to a single linear transformation $\mathbf{W}'\mathbf{x} + \mathbf{b}'$, eliminating the network’s ability to learn non-linear decision boundaries. All runs are logged to W&B.

B. Word2Vec Embeddings (Task 1b)

Word2Vec [6] is a self-supervised method that learns word representations by training a shallow neural network to predict surrounding words using the CBOW variant, which is the gensim default. The key insight is the distributional hypothesis, which states that words appearing in similar linguistic contexts tend to have similar meanings. After training, semantically similar words are placed nearby in the vector space, enabling the model to generalize across synonyms and related concepts.

For this project, Word2Vec is trained on the concatenation of all 18,846 preprocessed documents by combining the training and test token lists. This approach is permissible because Word2Vec is completely unsupervised since it uses no class labels, only word co-occurrence statistics. Combining both splits maximizes the vocabulary and provides more training signal for each word, leading to higher-quality embeddings.

Key hyperparameters: `vector_size=100` (standard for medium corpora), `window=5` (the recommended value for semantic representations), `min_count=2` (excludes rare tokens with insufficient context to form meaningful vectors), and `seed=42` (for reproducibility).

Two separate models are trained, one for 1 epoch and one for 10 epochs. The motivation is to empirically study how training duration affects the quality and structure of the learned embedding space, which is a direct requirement of Task 1b.

To obtain document-level representations, mean pooling is applied: in-vocabulary word vectors are averaged into a single 100-dimensional vector. Documents with no vocabulary tokens receive a zero vector. Mean pooling loses word order entirely, but is a well-established and computationally efficient baseline.

1) *Comparing 1 Epoch vs 10 Epochs*: The difference between training for 1 epoch versus 10 epochs is not just a matter of degree, it reflects a qualitative change in what the model has learned about language structure.

After just 1 epoch, each word has been seen in context only once. The vectors have barely moved from their random initialization. The model has not yet had enough exposure to word co-occurrence patterns to produce meaningful geometric relationships in the embedding space. This is visible in the PCA plot (Figure 1). After 1 epoch, the 200 most frequent words cluster tightly in the center of the 2D projection, with many outliers scattered seemingly at random. The layout does not reflect any meaningful semantic grouping.

After 10 epochs, the model has made 10 complete passes through all 18,846 documents, seeing each word in its full range of contexts. The vectors have converged toward positions that genuinely reflect how words are used. Words that co-occur frequently end up close together, while words from different topics separate out. In the PCA plot, the 10-epoch projection shows a much broader spread, with words moving to more distinct positions in the 2D space.

2) *Mean Cosine Shift Analysis*: To quantify how much the word vectors moved between the two models, cosine distance is computed for each of the 200 most frequent words:

$$d_{\cos}(\mathbf{a}, \mathbf{b}) = 1 - \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

A distance of 0 means the two vectors are identical, pointing in exactly the same direction in 100-dimensional space. A distance of 1 means the vectors are orthogonal, which means they are completely unrelated. A distance of 2 indicates they point in opposite directions.

The mean cosine shift across the 200 sampled words is **0.6876**, which is a large value. For context, if all vectors were random and unrelated, we would expect a mean shift close to 1.0 because they would be orthogonal on average. A mean shift of 0.6876 confirms that the vectors moved substantially. They are far from identical, but they also have not reversed direction. The vectors have genuinely repositioned in the 100-dimensional space to reflect the learned co-occurrence patterns.

The bar chart in Figure 2 shows the top 20 words with the largest shifts. Interestingly, many of the words that shifted the most are generic, polysemous words such as “new”, “call”, “free”, and “number”, which are words whose meaning varies greatly across different newsgroup topics. After 10 epochs, the model has encountered these words in many different contexts and adjusted their representations to better reflect their varied usage. Topic-specific words, like “hockey” or “space”, were

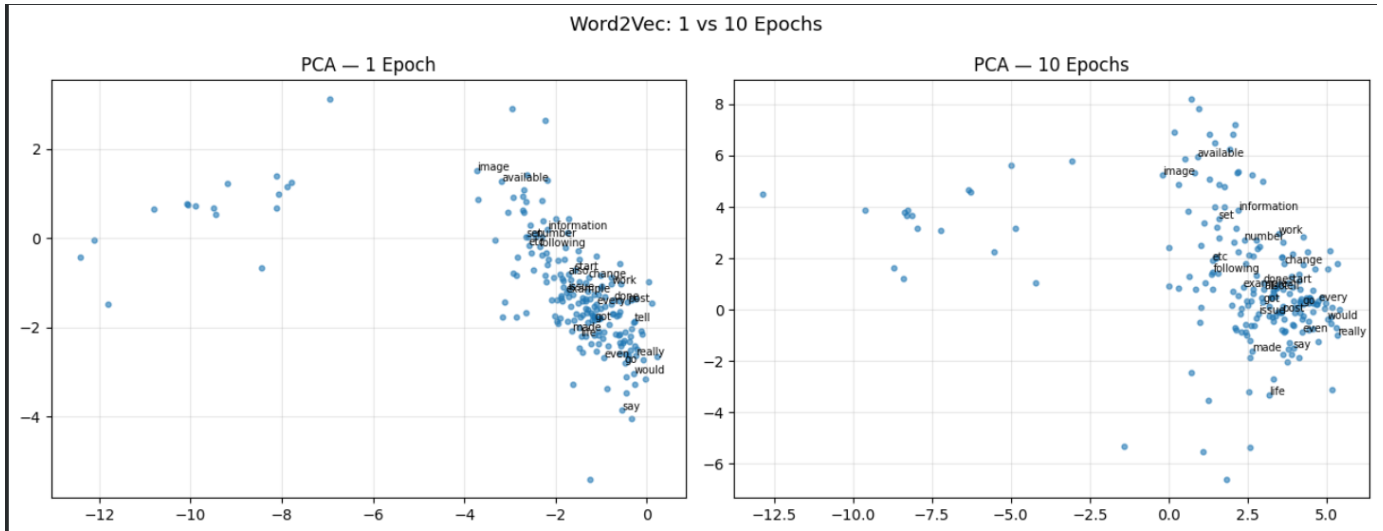


Fig. 1. PCA projection of the 200 most frequent Word2Vec word vectors after 1 epoch (left) and 10 epochs (right). After 1 epoch, words cluster near the origin with little structure. After 10 epochs, vectors spread distinctly, reflecting learned semantic relationships.

likely already converging faster because they appear in more consistent contexts.

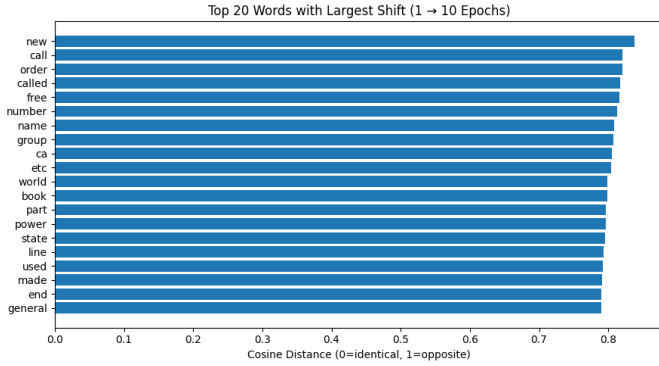


Fig. 2. Top 20 words with the largest positional shift between 1-epoch and 10-epoch Word2Vec models. Generic, polysemous words shift most, as their meaning varies across newsgroup topics. Mean cosine shift = 0.6876.

Despite these improved representations, the final classification accuracy of Word2Vec (0.586) remains modest. This reveals an important limitation of mean pooling. Even with high-quality 10-epoch vectors, averaging all word vectors in a document discards the relative importance of individual words. For example, a document about hockey that also contains many generic words like “think”, “really”, and “also” will have its distinctive words, such as “puck” and “goalie”, diluted by those generic terms in the mean-pooled vector.

C. TF-IDF Vectorization (Task 1c)

Term Frequency-Inverse Document Frequency (TF-IDF) [7] represents each document as a sparse vector of word importance scores. Unlike Word2Vec, which learns from co-occurrence patterns, TF-IDF computes a statistical weighting that directly captures how distinctive each word is to a specific document relative to the whole corpus.

For a word w in document d within a corpus of N documents:

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \log\left(\frac{N}{df(w)}\right)$$

where $\text{TF}(w, d)$ is the frequency of w in d and $df(w)$ is the number of documents containing w . A word that is frequent in this document but rare across the corpus gets a high score. A word that appears everywhere, like “the”, gets a low IDF score regardless of its local frequency.

The vectorizer is configured with `max_features=5000` (retaining the 5,000 most informative terms) and `sublinear_tf=True` (applying $\log(1 + \text{TF})$ to dampen very high term frequencies). `fit_transform` is applied to training data only; the test set uses `transform` to avoid information leakage through the IDF weights.

D. TF-IDF with Bigrams and LSA (Task 1d) — Experiment Design

Task 1d required designing one experiment that could potentially improve results while keeping the neural network architecture unchanged. After analysing the results of Tasks 1b and 1c, two hypotheses motivated the choice of TF-IDF with Bigrams and LSA:

Hypothesis 1: Plain TF-IDF loses phrase-level meaning by treating each word independently. Words like “not good” and “not bad” would produce similar unigram vectors, despite meaning very different things. By adding bigrams, the vectorizer captures two-word phrases as single features, preserving more semantic nuance.

Hypothesis 2: The 5,000-dimensional TF-IDF vector is sparse and high-dimensional, which may cause the neural network to overfit or fail to find useful structure. Applying LSA to compress these sparse vectors into a lower-dimensional dense space might produce a more informative, less noisy input.

Word2Vec + NN Training Curves (10-epoch embeddings)

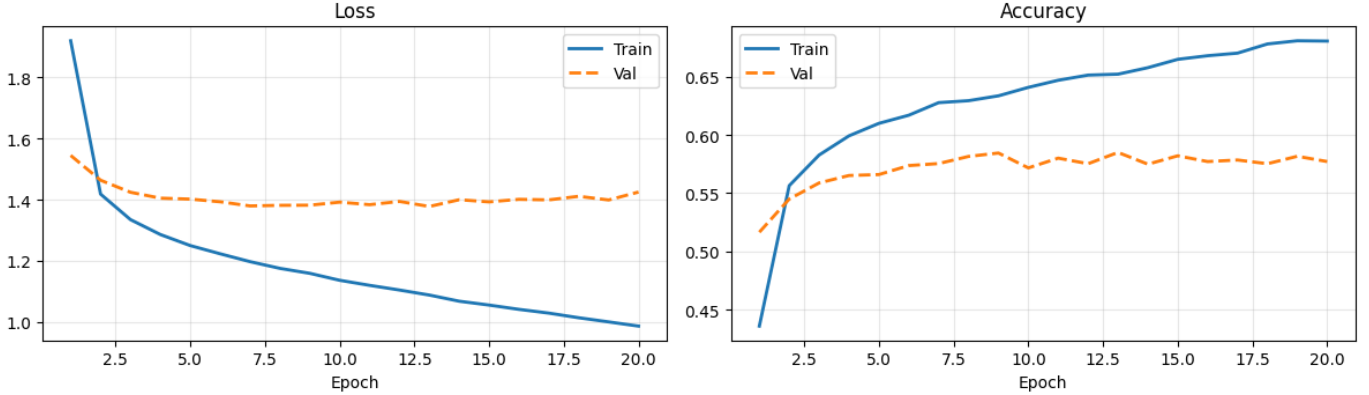


Fig. 3. Training and validation loss and accuracy curves for the Word2Vec + NN classifier (10-epoch embeddings) over 20 epochs. Validation accuracy plateaus around 0.58, consistent with the final test accuracy of 0.586.

TF-IDF + NN Training Curves

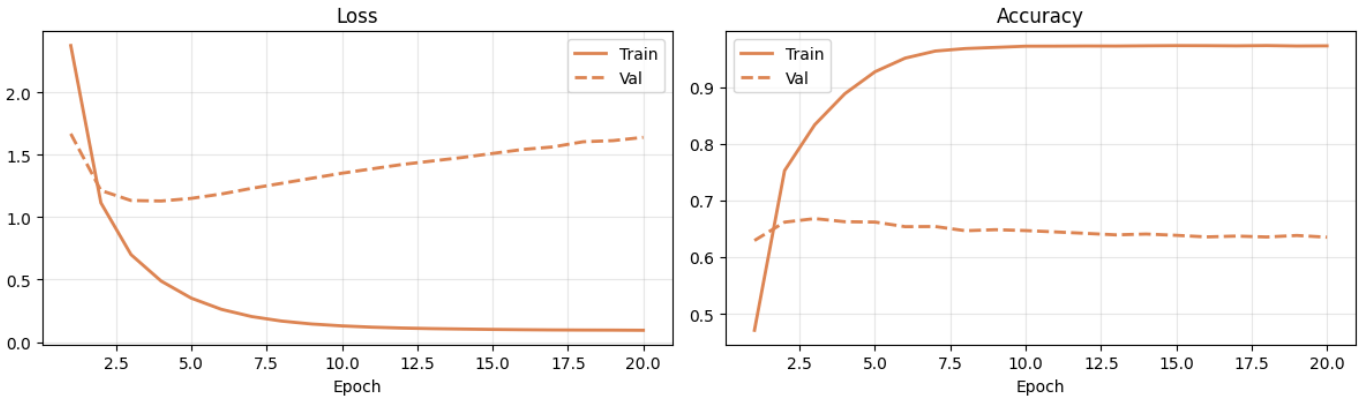


Fig. 4. Training and validation accuracy curves for the TF-IDF + NN classifier over 20 epochs. Training accuracy reaches 97.3% while validation accuracy plateaus at 63.6% after epoch 3, illustrating severe overfitting caused by the high-dimensional input relative to dataset size.

The implementation combines these two ideas in a pipeline. First, `ngram_range=(1, 2)` instructs the vectorizer to include both unigrams and bigrams as features. Expressions such as “hard drive”, “space shuttle”, and “not guilty” become single features, preserving meaning that unigram TF-IDF discards. The `max_features` parameter is increased to 20,000 to accommodate the expanded vocabulary.

Second, Latent Semantic Analysis (LSA) [8] is applied via `TruncatedSVD` with `n_components=300`, compressing the 20,000-dimensional sparse matrix into 300 dense dimensions. `TruncatedSVD` is used rather than standard PCA because PCA requires the full data matrix to be loaded into memory as a dense array, which is computationally infeasible at 20,000 dimensions with 11,000+ documents. `TruncatedSVD` operates directly on sparse matrices, making it practical. The 300 components retained 21.4% of total variance. The neural network input dimension becomes 300.

V. BERT-BASED METHODS

A. BERT Fine-Tuning (Task 2)

BERT (Bidirectional Encoder Representations from Transformers) [2] represents a fundamental shift in how language models are built. Rather than producing static word vectors, BERT is a deep transformer model that computes context-dependent representations by reading the entire input sequence simultaneously through self-attention. The word “bank” in “river bank” and “bank account” produces different BERT representations because the model considers all surrounding words when building each token’s embedding.

The `bert-base-uncased` variant is used, comprising 12 transformer layers, 12 self-attention heads, a hidden dimension of 768, and approximately 110 million parameters. It was pre-trained on 3.3 billion words from Wikipedia and BookCorpus using two self-supervised objectives: Masked Language Modeling (MLM, predicting randomly masked tokens) and Next Sentence Prediction (NSP, predicting whether two sentences are consecutive). This pretraining gives BERT deep knowledge

of English syntax, semantics, and world knowledge before any task-specific training.

For classification, `BertForSequenceClassification` is instantiated with `num_labels=20`. This adds a single linear classification head on top of the representation produced by the special `[CLS]` token. This token is prepended to every input sequence and, after passing through all 12 transformer layers where it attends to every other token, accumulates a summary of the entire document. The classification head maps this 768-dimensional `[CLS]` vector to 20 class logits.

The `BertTokenizer` uses the WordPiece subword algorithm, which splits rare or complex words into smaller units. For example, “bioelectromagnetics” might be tokenized as `["bio", "##electro", "##magnet", "##ics"]`. This ensures that BERT can handle any English word without encountering out-of-vocabulary tokens, unlike Word2Vec which must skip unknown words entirely. A maximum sequence length of 128 tokens is enforced, with padding and attention masking for shorter sequences.

The training configuration uses AdamW with **LR = 2e-5** — deliberately 50× smaller than the Task 1 rate to avoid destroying BERT’s pretrained representations with large early gradient updates. A **linear warmup scheduler** ramps the learning rate from zero to 2×10^{-5} over the first 10% of steps (212 steps), protecting the pretrained encoder from the large gradients produced by the randomly initialized classification head at the start of training. **Gradient clipping** (`clip_grad_norm=1.0`) prevents catastrophically large updates from individual batches. **Batch size = 16** is used because each BERT forward pass over 12 transformer layers is far more expensive than the Task 1 matrix multiplications. Training runs for **3 epochs**, sufficient given BERT’s strong pretrained initialization; longer training risks overfitting with 110M parameters and only 11,314 training documents.

B. BERT with Masked Language Modeling (Task 3)

Task 3 implements domain-adaptive pretraining [10], which is a two-stage approach that adds an intermediate adaptation step between the original BERT pretraining and the final classification fine-tuning.

The motivation is straightforward because standard BERT was pretrained on Wikipedia and BookCorpus, which consist of formal and carefully edited text. Newsgroup posts are fundamentally different in character. They contain technical abbreviations, hardware model numbers, internet slang, heated political discussion, and the informal discourse of online communities in the early 1990s. BERT’s internal representations are calibrated to the vocabulary and style of its pretraining data. When fine-tuned directly on newsgroup data, it must simultaneously adapt its language understanding and learn the classification task, which is a harder joint problem.

The solution is to first expose BERT to newsgroup text through continued MLM pretraining, before any classification labels are introduced. This allows the model to adjust its vocabulary representations toward the domain-specific language without the constraint of the classification objective.

Stage 1 — MLM Pretraining: `BertForMaskedLM` is fine-tuned on all 18,846 newsgroup documents by combining the training and test sets since no labels are used for 2 epochs. The `DataCollatorForLanguageModeling` randomly masks 15% of tokens per sequence, replacing them with `[MASK]`. The model is trained to predict the original tokens from context alone. Since no class labels are involved, this is self-supervised learning where the text teaches the model about its own structure.

The masking rate of 15% follows the original BERT paper and has proven robust across many applications. If the rate were lower, such as 5%, the task would be too easy and the model would learn little. If it were higher, such as 40%, too much context would be removed, making the task nearly impossible and training unstable.

The MLM loss decreases from 2.4558 in epoch 1 to 2.2969 in epoch 2, confirming successful vocabulary adaptation. The optimizer uses `MLM_LR=5e-5`, which is slightly higher than the classification learning rate because the MLM objective is more stable and less prone to catastrophic interference than classification fine-tuning on a new task.

Only 2 MLM epochs are used rather than more. This choice is deliberate because too many MLM epochs risk catastrophic forgetting, which is a phenomenon where the model over-adapts to the newsgroup domain and loses the general English language understanding encoded in the original BERT weights. The goal is domain adaptation rather than domain replacement, and 2 epochs achieves this balance.

Stage 2 — Classification: The MLM-adapted weights are loaded into `BertForSequenceClassification` using `ignore_mismatched_sizes=True`. This parameter is necessary because the MLM model’s output head predicts over the full 30,000-token vocabulary, whereas the classification head outputs 20 class scores. The parameter tells PyTorch to ignore the size mismatch and randomly initialize the new classification head while loading all the shared encoder weights. Classification fine-tuning then proceeds identically to Task 2.

VI. LLAMA-3 EXPERIMENTS

A. Zero-Shot and Few-Shot Classification (Task 4)

Llama-3-8B-Instruct [3] is a large autoregressive language model with 8 billion parameters, instruction-tuned to follow natural language prompts. Unlike BERT, which encodes a fixed-length input and produces a classification label, Llama-3 generates text token by token. To use it for classification, the model is prompted to output a category name, and the response is matched to one of the 20 known class names using substring matching.

The model is loaded in 4-bit quantization using `BitsAndBytesConfig` with `load_in_4bit=True` and `bnb_4bit_compute_dtype=torch.float16`. Without quantization, Llama-3-8B requires approximately 16GB of VRAM in float16 precision, which exceeds the 15GB available on a single T4 GPU. 4-bit quantization represents each weight using only 4 bits rather than 16. This reduces memory to approximately 6GB with minimal

BERT Fine-tuning Curves

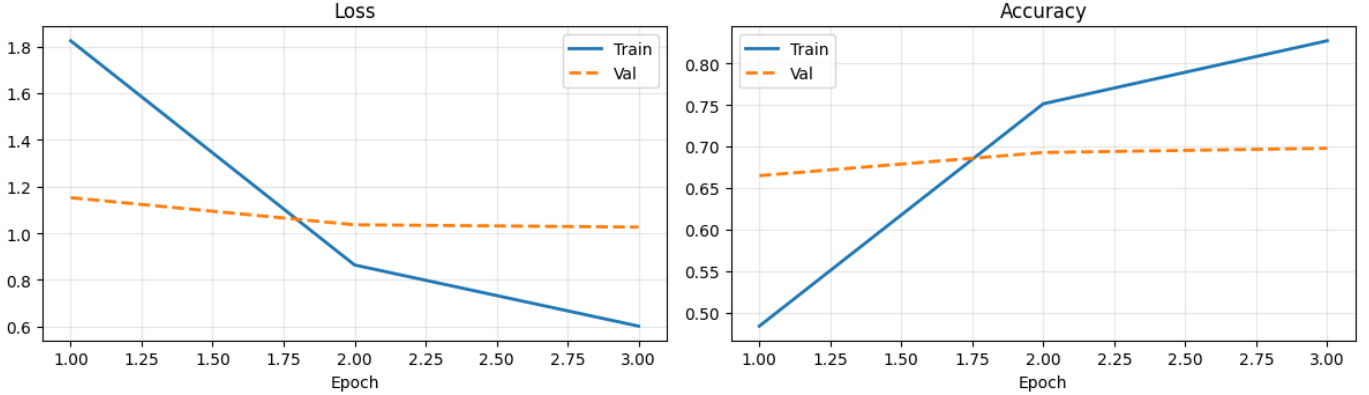


Fig. 5. Training and validation loss (left) and accuracy (right) for BERT fine-tuning over 3 epochs. Training accuracy reaches 82.7% while validation accuracy plateaus at 69.8%, indicating mild overfitting that stabilizes after epoch 2.

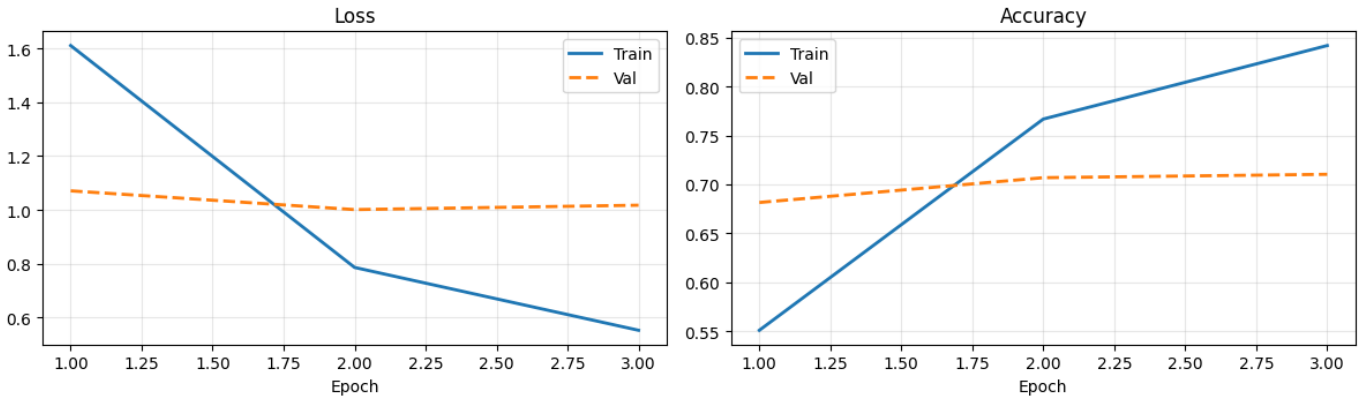
BERT (MLM $\rightarrow$ Classification) Training Curves

Fig. 6. Training and validation curves for BERT MLM pretraining followed by classification fine-tuning. Starting from domain-adapted weights, the model converges faster and reaches 0.710 test accuracy, which is 0.012 above Task 2.

accuracy loss. The bitsandbytes library uses block-wise quantization with double quantization of the scale factors to minimize error.

Due to the computational cost of autoregressive inference, which means generating one token at a time through the full 8-billion parameter network, evaluation is conducted on a random sample of $N_{\text{eval}} = 200$ test documents with `seed=42` for reproducibility. The `max_new_tokens=20` parameter limits the output length. Since we only want a category name, 20 tokens is sufficient and prevents runaway generation.

Zero-shot: No training examples are provided. The prompt includes the full list of 20 class names and the first 300 characters of the document, with the instruction to respond with only the category name. The model achieved 0.255 accuracy, which is $5.1\times$ higher than random guessing ($1/20 = 0.05$). This demonstrates that Llama-3 possesses relevant world knowledge. For example, it understands that “puck” and “goalie” relate to hockey, but it cannot reliably differentiate among all 20 fine-grained categories when many share vocabulary

or require nuanced topic understanding.

Few-shot: One labeled training example per class for a total of 20 examples is prepended to each prompt. Each example consists of a short document excerpt of 200 characters and its correct label, forming a pattern for the model to follow. An important implementation constraint arose here: the initial design of 20 examples per class exceeded the 512-token context limit, causing prompt truncation. The final design uses one example per class with 200-character excerpts to stay within the context window. The model achieved 0.430 accuracy, which is an improvement of 0.175 over zero-shot. This substantial gain is achieved without any weight updates, happening purely through the information provided in the prompt context. It demonstrates that Llama-3 is capable of rapid in-context adaptation when given clear examples.

B. QLoRA Fine-Tuning (Bonus)

QLoRA (Quantized Low-Rank Adaptation) [4] enables parameter-efficient fine-tuning of large models by combining

4-bit quantization of the base model with small trainable adapter matrices. The key insight of LoRA [16] is that fine-tuning updates to large weight matrices tend to have a low intrinsic rank. Meaningful adaptation can be captured by breaking down the weight update ΔW into the product of two small matrices, $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with a rank r that is much smaller than the original dimensions.

In this implementation, LoRA adapters are added to the query and value projection matrices of all transformer attention layers by setting `target_modules=["q_proj", "v_proj"]` with a rank $r = 8$ and a scaling factor of `lora_alpha=16`. With these settings, only about 3.4 million parameters are trainable out of 8 billion total, which is approximately 0.04%. The base model weights remain frozen and in 4-bit precision throughout training.

Training uses the TRL library’s `SFTTrainer` with 2,000 randomly sampled training examples. Each example is formatted as an instruction prompt containing the 20 class names, the document text truncated to 300 characters, and the correct category name as the target. Training runs for 1 epoch with `learning_rate=2e-4`. This learning rate is higher than BERT’s value because the LoRA adapters are randomly initialized and learn from scratch, so they need a larger step size to converge meaningfully in just 1 epoch. The base model’s frozen weights do not need protection from large updates because they are not being modified. The model achieved 0.645 accuracy.

VII. RESULTS AND DISCUSSION

A. Quantitative Results

Table I summarizes the test accuracy achieved by all eight experiments. Llama-3 experiments use a 200-sample random subset, while all other models use the full 7,532-sample test set.

TABLE I
CLASSIFICATION ACCURACY — ALL EXPERIMENTS

Method	Accuracy	Input Dim
Word2Vec (10 ep) + NN	0.586	100
TF-IDF + NN	0.636	5,000
TF-IDF + Bigrams + LSA + NN	0.615	300
BERT Fine-tuning	0.698	768 (CLS)
BERT + MLM + Fine-tuning	0.710	768 (CLS)
Llama-3 Zero-Shot	0.255	N/A
Llama-3 Few-Shot	0.430	N/A
Llama-3 QLoRA (Bonus)	0.645	N/A

B. Why TF-IDF Outperforms Word2Vec

The result that TF-IDF (0.636) outperforms Word2Vec (0.586) appears counterintuitive. Word2Vec is a more sophisticated method that learns semantic relationships between words, while TF-IDF is a statistical weighting with no notion of word meaning.

The explanation lies in the nature of the 20 News-groups classification task. Each category is defined primarily by distinctive, domain-specific vocabulary. Words such

as “puck”, “goalie”, and “powerplay” unambiguously signal `rec.sport.hockey`. The term “encryption” strongly suggests `sci.crypt`, and the phrase “space shuttle” points clearly to `sci.space`. These words appear frequently within their categories but rarely elsewhere. This gives them high TF-IDF scores that make them standout features in the document vectors.

Word2Vec represents each document as the mean of all its word vectors. This averaging dilutes the strong category-specific signals. For example, the distinctively important term “encryption” in a `sci.crypt` document is averaged with generic terms like “really”, “think”, “also”, and “good”, which are common across all categories. The resulting mean-pooled vector is pulled away from the topic-specific direction by these uninformative words, reducing category separability.

Overfitting is another major factor. The TF-IDF model’s training accuracy reaches 97.3% while test accuracy plateaus at 63.6% after epoch 3, which is a clear overfitting pattern observable in the W&B training curves. This occurs because the first linear layer has $5000 \times 256 = 1.28$ million learnable weights, while the training set contains only 11,314 samples. The model has far more capacity than needed and memorizes training examples rather than learning generalizable patterns. Despite this issue, TF-IDF still outperforms Word2Vec because the early representations are highly discriminative.

The Word2Vec model shows more gradual convergence with less extreme overfitting. With only 100-dimensional input, the first layer has $100 \times 256 = 25,600$ weights, which is a much more balanced ratio relative to the training set size. Validation accuracy improves slowly and steadily but never reaches TF-IDF’s peak because the mean-pooled vectors are fundamentally less informative for this specific task.

C. Why LSA Did Not Improve TF-IDF

The TF-IDF + Bigrams + LSA experiment (0.615) was designed with the hypothesis that bigrams and semantic compression would improve upon plain TF-IDF. Instead, the result was a decrease of 0.021 accuracy points. Two factors explain this outcome:

First, SVD compression from 20,000 to 300 dimensions retains only 21.4% of the total variance in the TF-IDF matrix. The remaining 78.6% of variance, which includes many discriminative features, is discarded. For topic classification where specific vocabulary is highly predictive, losing most of the feature variance is costly.

Second, LSA compression forces words from different topics into shared latent dimensions. Categories that share vocabulary but differ in topic, such as `sci.crypt` and `talk.politics.misc` which both discuss encryption and government, may have their representations conflated by LSA, making them harder to separate.

The bigrams themselves add value because expressions like “space shuttle” and “hard drive” are stronger topic signals than their constituent unigrams. However, the information loss from SVD outweighs this benefit at 300 components. Increasing to 500 or 1,000 components would retain more variance and

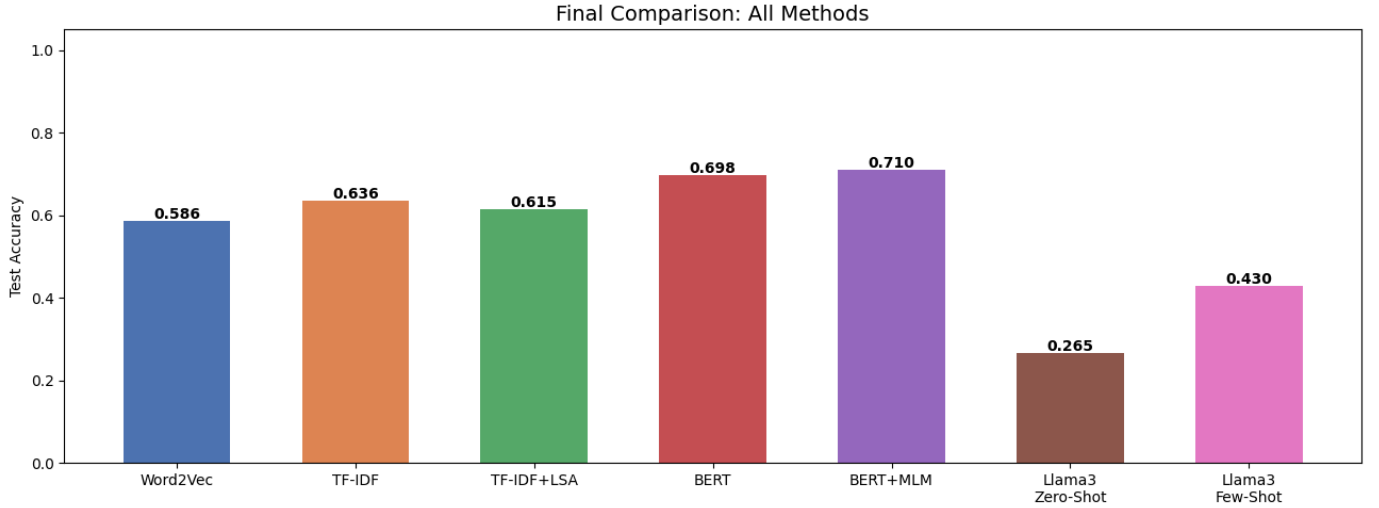


Fig. 7. Test accuracy across all eight experiments. BERT+MLM achieves the highest overall accuracy at 0.710. The gap between Llama-3 zero-shot (0.255) and fine-tuned models confirms that task-specific training is critical.

might improve the result, but at the cost of computational efficiency. LSA is not universally beneficial and must be tuned carefully for each task.

Future work should test a higher number of SVD components, such as 500 or 1,000, to see whether more retained variance closes the gap with plain TF-IDF. Evaluating TF-IDF with bigrams without LSA as an intermediate step would also isolate the contributions of bigrams and compression independently.

D. BERT Fine-Tuning Analysis

BERT fine-tuning (0.698) achieves a 0.062 improvement over the best Task 1 result. This gain is attributable to three fundamental architectural advantages:

First, BERT uses contextual representations. Every token’s embedding is computed in the context of all surrounding tokens. The word “apple” in a computer hardware post produces a different BERT embedding than in a food post. This context-sensitivity is impossible with Word2Vec’s static embeddings.

Second, the architecture performs full sequence encoding. BERT processes up to 128 tokens simultaneously without any aggregation loss. While mean-pooling in Word2Vec flattens the entire document into a single vector, BERT’s [CLS] representation is built through 12 layers of attention over the full sequence, preserving deep document-level information.

Third, the approach profits from transfer learning. BERT starts from 110 million pretrained parameters that already encode English syntax, semantics, and world knowledge. The task-specific fine-tuning merely adapts this existing knowledge to the classification objective, requiring only 3 epochs to converge.

The per-class performance reveals a clear pattern across all experiments. Categories with distinctive, low-ambiguity vocabulary achieve high F1 scores, such as `rec.sport.hockey` at 0.88 to 0.89, `misc.forsale`

at 0.83 to 0.84, and `sci.med` at 0.83 to 0.84. Conversely, categories with overlapping vocabulary score substantially lower, including `talk.religion.misc` at 0.09 to 0.20, `alt.atheism` at 0.48 to 0.49, and `talk.politics.misc` at 0.49 to 0.50. This pattern suggests that the remaining performance gap is driven by the inherent ambiguity in the data itself rather than model limitations.

E. BERT + MLM Pretraining Analysis

The domain-adaptive MLM stage improved accuracy from 0.698 to 0.710. While modest in absolute terms, this improvement is achieved at zero additional labeling cost because only unlabeled newsgroup text is needed for MLM pretraining. This makes it a practical enhancement for real-world NLP pipelines where labeled data is scarce but unlabeled domain text is abundant.

The mechanism is visible in the training curves (Figure 6). The MLM-initialized model converges faster in the classification stage, reaching higher validation accuracy by epoch 1 than the standard BERT model. This faster convergence reflects a better initialization since the model’s vocabulary representations are already adapted to newsgroup language, meaning less of the fine-tuning budget is spent on domain adaptation.

F. Llama-3 Analysis

The Llama-3 experiments reveal an instructive pattern. An 8-billion parameter model achieves only 0.255 in zero-shot classification, which is substantially below the 0.636 achieved by TF-IDF, a purely statistical method.

This result demonstrates that general-purpose language understanding is not the same as task-specific classification capability. Llama-3 was trained to generate coherent text rather than to reliably discriminate among 20 fine-grained newsgroup categories. However, the model demonstrates genuine

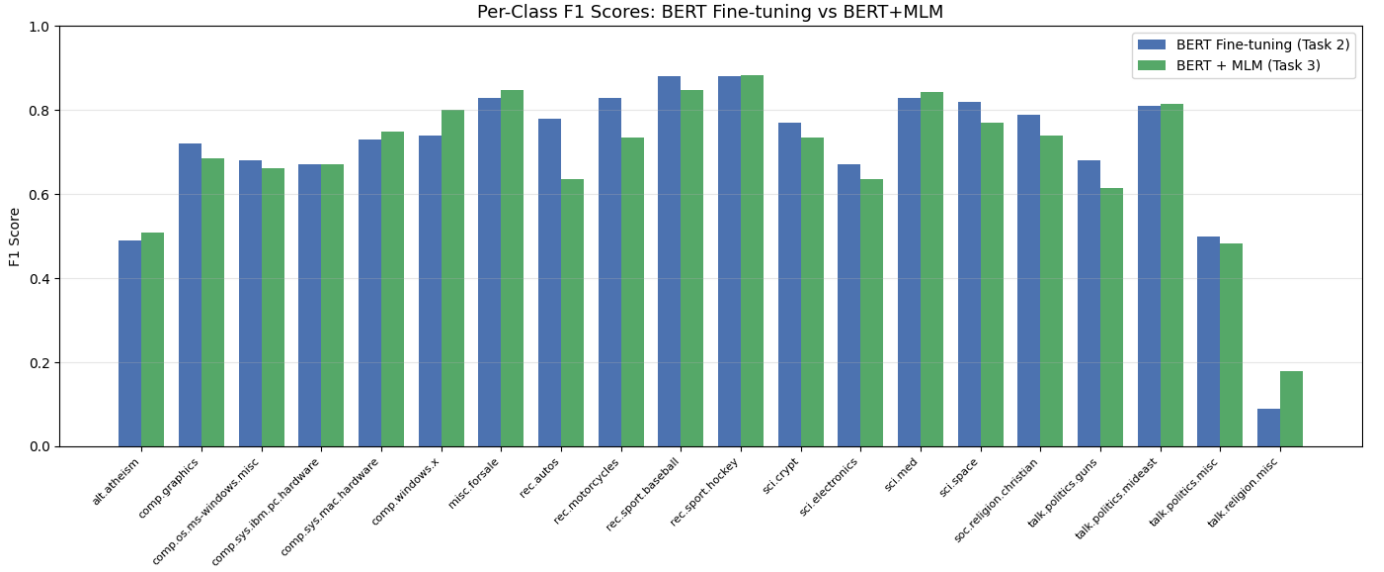


Fig. 8. Per-class F1 scores for BERT fine-tuning (Task 2) and BERT+MLM (Task 3). Categories with distinctive vocabulary (`rec.sport.hockey`, `misc.forsale`) score consistently high, while semantically overlapping categories (`talk.religion.misc`, `alt.atheism`) score substantially lower across both models.

topic understanding by performing $5.1\times$ better than random guessing, with most errors occurring on the most ambiguous categories.

The few-shot improvement from 0.255 to 0.430 is striking because it requires no training weights to change. Simply showing the model one example per class in the prompt reshapes its output distribution toward the task. This in-context learning capability is a major advantage of large language models.

The QLoRA result (0.645) is the most effective of the three Llama-3 evaluations. By training only 3.4 million parameters for a single epoch on 2,000 examples, the model surpasses all Task 1 baselines and approaches BERT performance. This demonstrates the efficiency of LoRA-based adaptation and suggests that with more training data and more epochs, the QLoRA model could challenge BERT+MLM.

VIII. PARAMETER ANALYSIS AND POTENTIAL EXPERIMENTS

A. Task 1 — Neural Network Parameters

The most impactful parameters are the hidden dimension of 256 and the learning rate of 0.001. If the hidden dimension were reduced to 64, the model would have fewer parameters, dropping from 25,600 to 6,400 weights in the first layer for Word2Vec. This would lower overfitting risk but also reduce representational capacity. For TF-IDF, where overfitting is the dominant failure mode, a smaller hidden dimension might improve test accuracy.

The learning rate of 0.001 was not tuned. A sweep over values like 0.0001 and 0.01 might identify a better setting. For TF-IDF, a lower learning rate combined with early stopping after epoch 3, where validation accuracy peaks, would likely improve the final test accuracy.

Adding Dropout (0.3) between the two linear layers would provide regularization by randomly zeroing 30% of activations during training. This forces the network to learn redundant representations, which is useful for TF-IDF where the network has excessive capacity relative to the dataset size.

For Word2Vec, increasing the `vector_size` from 100 to 300 would produce richer word representations. This would increase the parameter matrix size in the first layer and slightly slow down training, but it typically improves vector quality and raises classification accuracy by a few percentage points.

B. Task 2 — BERT Parameters

The most impactful untuned parameter is the maximum sequence length of 128. Many newsgroup posts exceed 128 tokens, and truncation discards the later portions of documents that may contain key topical content. Increasing this value to 256 would capture more context at the cost of longer training time, since attention complexity scales quadratically with sequence length. A longer sequence length combined with a slightly lower learning rate like 10^{-5} might improve generalization.

Training for 4 to 5 epochs with early stopping based on validation loss was not explored. This could improve accuracy since training performance was still rising at epoch 3 while validation accuracy had stabilized.

C. Task 3 — MLM Parameters

The masking probability of 15% follows the original BERT setup and is well-established. Increasing the number of MLM epochs from 2 to 3 or 4 would provide more domain adaptation but risks catastrophic forgetting. The model might over-adapt to newsgroup text and degrade its general English language

representations. The current 2-epoch setting achieves a measurable improvement without this risk, suggesting it is close to optimal.

An alternative experiment would be to use a higher learning rate for MLM, such as 10^{-4} , to accelerate adaptation, combined with a lower classification fine-tuning learning rate to prevent destroying the MLM-adapted weights.

D. Task 4 — Llama-3 Parameters

For QLoRA, the most impactful change would be training on the full 11,314 training examples for 3 epochs instead of 2,000 examples for 1 epoch. This would provide significantly more gradient updates, likely pushing accuracy well above 0.645 and closer to BERT+MLM levels.

Increasing the LoRA rank from $r = 8$ to $r = 16$ or $r = 32$ would increase the number of trainable parameters, providing more adapter capacity. However, for a 1-epoch, 2,000-sample training setup, the current rank is sufficient because the main bottleneck is the number of training examples rather than adapter size.

For the few-shot setup, a practical improvement would be to reduce the examples per class to 3 or 5 and increase the maximum length parameter to 1024. This change would prevent prompt truncation while still providing useful context, closing the gap with QLoRA performance.

IX. REFLECTIONS AND LEARNINGS

This project provided several broader insights relevant to applied NLP system design:

- **Representation is the bottleneck:** The information content of the input representation determines the performance ceiling. A more powerful classifier cannot recover information that the representation discards. This suggests that in practice, effort is better spent improving data representations than tuning classifier architecture.
- **Dataset properties determine method suitability:** TF-IDF’s advantage over Word2Vec is specific to datasets where categories are defined by distinctive terminology. On tasks requiring synonym understanding or paraphrase detection, this ranking would likely reverse. Choosing a vectorization method requires understanding the discriminative structure of the target dataset, not just general method rankings from the literature.
- **Domain adaptation is low-cost and reliable:** Continued MLM pretraining on target-domain text requires no additional labels and imposes minimal computational overhead. This makes it a practical default step for any NLP pipeline being deployed on text that differs stylistically from standard pretraining corpora.
- **Context window management is a core engineering concern:** When deploying LLMs for classification, the context budget must be designed explicitly. Prompt components, examples, and document text compete for a fixed token limit. Exceeding this limit silently truncates input and degrades performance without error messages, making it an easy source of undetected bugs.

- **Parameter-efficient fine-tuning changes the cost calculus for large models:** Adapter methods like QLoRA decouple model scale from fine-tuning cost. A practitioner with limited compute can now adapt an 8-billion parameter model on a single GPU. The relevant question is no longer whether to use a large model, but how much task-specific data is available.

X. CONCLUSION

This study demonstrates that for fine-grained 20-class topic classification, the quality and type of text representation is the primary determinant of performance, far more so than model complexity or scale. BERT-based methods achieve the highest accuracy, ranging from 0.698 to 0.710, by combining pre-trained contextual language understanding with task-specific fine-tuning. Domain-adaptive MLM pretraining provides a meaningful and free performance improvement by adapting BERT’s internal representations to newsgroup-specific vocabulary before classification training begins.

Among the Task 1 methods, TF-IDF (0.636) counterintuitively outperforms Word2Vec (0.586). This result is explained by the strong discriminative power of topic-specific vocabulary in this dataset, which TF-IDF captures precisely while Word2Vec’s mean-pooling dilutes the signal. The LSA compression experiment (0.615) confirms that dimensionality reduction can reduce discriminability when the retained variance is insufficient, as only 21.4% was preserved in this case.

The Llama-3 experiments reveal a crucial finding: a model with 8 billion parameters achieves only 0.255 accuracy in zero-shot classification, which is far below TF-IDF at 0.636. Few-shot prompting (0.430) and QLoRA fine-tuning (0.645) progressively close this gap, demonstrating that task-specific adaptation is necessary to harness LLM capabilities for structured classification tasks.

Future work should explore increasing BERT’s `max_length` parameter to 256 for richer document encoding, and extending QLoRA training to the full 11,314-sample training set for 3 epochs. Additional directions include applying `bert-large` with 24 layers and 340 million parameters for a higher baseline, as well as testing ensemble methods that combine TF-IDF’s vocabulary precision with BERT’s contextual understanding.

REFERENCES

- [1] K. Lang, “Newsweeder: Learning to filter netnews,” in *Proc. 12th Int. Conf. Machine Learning (ICML)*, 1995, pp. 331–339.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [3] Meta AI, “Meta Llama 3,” 2024. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3/>
- [4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” *Adv. Neural Inf. Process. Syst.*, vol. 36, 2024.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” in *Proc. ICLR*, 2013.

- [7] K. Sparck Jones, "A statistical interpretation of term specificity and its application in retrieval," *J. Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [8] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, "Indexing by latent semantic analysis," *J. Amer. Soc. Inf. Sci.*, vol. 41, no. 6, pp. 391–407, 1990.
- [9] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.
- [10] S. Gururangan *et al.*, "Don't stop pretraining: Adapt language models to domains and tasks," in *Proc. ACL*, 2020, pp. 8342–8360.
- [11] A. McCallum and K. Nigam, "A comparison of event models for naive Bayes text classification," in *Proc. AAAI Workshop Learning Text Categorization*, 1998, pp. 41–48.
- [12] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Proc. ECML*, 1998, pp. 137–142.
- [13] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *Proc. EMNLP*, 2014, pp. 1532–1543.
- [14] A. Vaswani *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [15] T. B. Brown *et al.*, "Language models are few-shot learners," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020.
- [16] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.